SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)

(Established under section 3 of the UGC Act 1956)

Re-accredited by NAAC with 'A' Grade

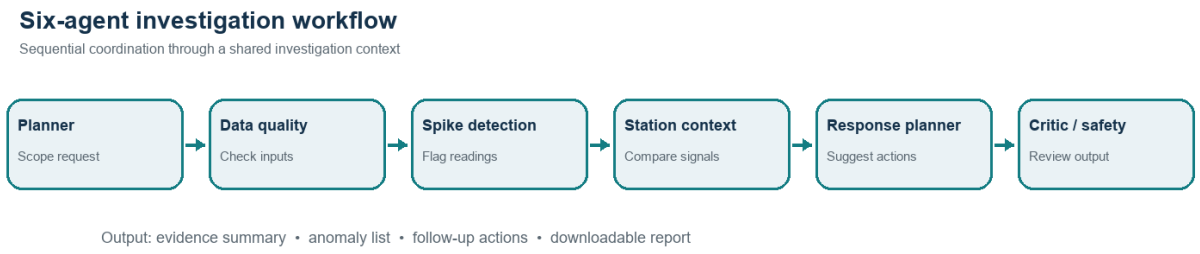
Founder: Prof. Dr. S. B. Mujumdar, M.Sc., Ph.D. (Awarded Padma Bhushan and Padma Shri by President of India)

BREATHEOPS AI

Multi-Agent Air Quality Investigation Centre

Technical Project Report and Proof of Concept Evaluation

A deterministic agent workflow for identifying unusual pollution readings and guiding human investigation



Project workflow overview

Prepared by

Prince Kumar | PRN 24070126140

Ishan Sharma | PRN 24070126073

Flexi-Credit Course 26–27 • CA3

Executive Summary

BreatheOps AI is a Streamlit proof of concept that investigates unusually high air-quality readings. It combines input validation, a robust statistical detector, six cooperating Python agents, cross-station and weather context, human-oriented follow-up actions, and downloadable evidence. The default demonstration analyzes synthetic hourly PM2.5 observations and flags readings for review; it does not declare a regulatory exceedance or identify a pollution source.

The reproducible seed-42 demo contains 1,344 station-hour rows across four stations. At Aotizhongxin, the selected pollutant has 331 valid observations, a median baseline of 41.4 μg/m^3 and an implemented upper threshold of 94.77 μg/m^3. The detector flags 9 readings; the observed peak is 365.5 μg/m^3.

Project dimension	Current implementation
Problem	Manually reviewing long monitoring time series can make unusual episodes difficult to find and contextualize.
Approach	Median plus k times scaled MAD for one-sided high-spike detection; default k = 3.
Agent design	Six deterministic Python agents execute in a fixed, evidence-sharing sequence.
User interface	Streamlit controls, Plotly chart, findings, agent logs, recommendations, CSV and report downloads.
Evidence	Reproducible synthetic scenario, method comparison, sensitivity analysis, test results and limitations.

Key conclusion

The project demonstrates a transparent, offline multi-agent analytics workflow. It is not an LLM-based system and does not compare trained AI models. The reported method comparisons are statistical baselines computed on the same synthetic readings. Results are appropriate for a classroom PoC and require human review before any operational use.

1. Problem and Project Objectives

1.1 Business problem

Air-quality monitoring stations produce frequent readings across several pollutants. Analysts may need to inspect long time series, separate ordinary variation from sharp excursions, check whether nearby stations show similar changes, review weather conditions, and communicate the evidence. Manual triage can be slow and inconsistent, especially when many station files are involved.

1.2 Proposed solution

BreatheOps AI automates the first-pass investigation. It validates the uploaded or demo data, establishes a station-level baseline, flags unusually high readings, checks other stations and weather variables where available, and assembles a cautious action list. The product is a decision-support PoC: a human should verify sensor quality, context, and source data before acting.

1.3 Objectives and scope

- Make anomaly rules visible and explainable to a reviewer.
- Keep input-quality warnings connected to the analysis result.
- Use multiple specialized agents to divide a single investigation into understandable steps.
- Provide station and weather context where the selected data source supports it.
- Export the evidence so a reviewer can inspect the flagged readings.

Out of scope: official AQI calculation, regulatory compliance decisions, source attribution, real-time alert delivery, medical advice, and autonomous public-safety action.

1.4 Data used

The project bundles the Beijing Multi-Site Air Quality dataset distributed by the UCI Machine Learning Repository. The checked-in archive contains 12 station CSV files with 35,064 hourly records per station, totaling 420,768 records from 1 March 2013 through 28 February 2017. It includes six pollutant variables (PM2.5, PM10, SO2, NO2, CO and O3) and six meteorological variables (temperature, pressure, dew point, rain, wind direction and wind speed). Missing values are present; in the bundled files, pollutant missingness ranges from about 1.5% to 4.9% by field. [1]

The app offers the local real-data option as one station file at a time. Therefore, this interface does not run a single 12-station joint investigation on the complete archive. The built-in synthetic option instead supplies four stations and 14 days for a reproducible multi-station workflow. Numerical results in Sections 3 and 4 are from that synthetic scenario; no anomaly count or accuracy score is claimed here for the complete UCI archive.

Data source	Use in PoC	Important caveat
Synthetic demo (seed 42)	Four stations × 336 hourly records = 1,344 rows; used for deterministic multi-station demonstration.	Contains a deliberately injected event; not a real-world accuracy benchmark.
UCI Beijing Multi-Site archive	12 CSV station files × 35,064 rows = 420,768 hourly records, 2013–2017.	Current UI selects one real station file at a time; full archive multi-station run is not exposed.
User-uploaded CSV	Supports a user file containing timestamp/date fields, pollutants and optionally station/weather fields.	Check units, station completeness, data continuity and source quality.

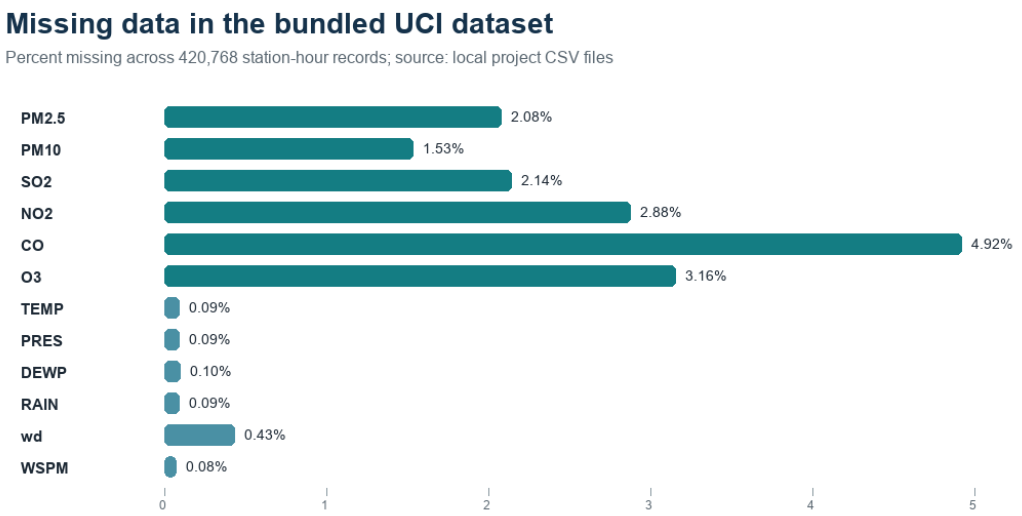


Figure 5. Missingness profile calculated from the 12 bundled station CSVs; pollutant fields have the largest gaps.

1.5 Planned scope and implementation status

Project requirement	Status in current project	Accurate description
Six specialist agents	Implemented as deterministic Python agents.	They run in a fixed sequence through a shared context; there is no LLM selecting tools or creating agents dynamically.
Pollution spike detection	Implemented for unusually high readings.	Uses median plus k × scaled MAD; default k = 3; high-side statistical flags only.
Multi-station context	Partial.	Works when multiple station records are present (including built-in demo); the local UCI selector loads one station file, and comparisons are not GIS-distance based.
Weather context	Partial, based on available columns and event window.	Summarizes measured weather fields; no validated atmospheric transport model.
Advisory and critic review	Implemented as rule-based recommendations and checks.	Advisory is for human review; “VALIDATED” is an internal status, not independent verification.
Map-style station view	Not present in the reviewed UI.	The app provides station selection, time-series visualization, summaries and context; describe a map as future work unless one is added.
UCI full-dataset analysis	Data files are included, but full joint analysis is not exposed by current UI.	Real-data UI analysis is one station file at a time; do not present synthetic figures as UCI findings.

2. System Architecture and Agent Workflow

The Streamlit interface gathers the source, station, pollutant, time window, and threshold sensitivity. The orchestrator builds a shared investigation context and runs each specialist agent in order. Outputs include a structured finding, chart, anomaly rows, recommendations, an audit status, and report downloads.

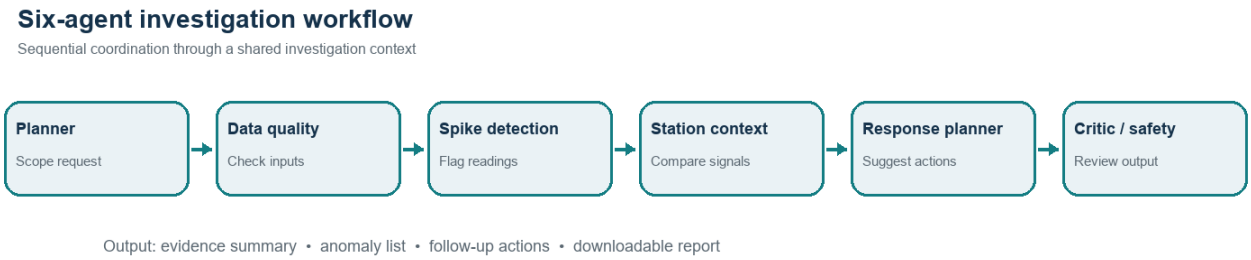


Figure 1. Fixed, sequential orchestration; the shared context carries evidence between agents.

Agent	Responsibility	Typical output
Planner	Interprets selected analysis target and input schema.	Investigation plan and required checks.
Data Quality	Checks usable rows, timestamp and missingness conditions.	Quality status and warnings.
Spike Detection	Calculates baseline, spread and high-spike threshold.	Threshold, anomaly count and flagged rows.
Multi-Station Context	Compares available station observations around the event and summarizes weather.	Provisional spatial label and weather context.
Response Planner	Maps findings to proportionate follow-up suggestions.	Inspection, calibration or expert-review actions.
Critic and Safety	Applies output checks and attaches a validation status.	Validation notes and reviewed status.

2.1 Tools and data flow

The project uses Python, pandas, NumPy, Streamlit, and Plotly. Agent classes are ordinary Python code connected by the orchestrator; they are not independent model endpoints. No source scan found LLM/API calls. The system works locally and can use generated demo data without a paid API.

3. Detection Method and Statistical Comparison

3.1 Implemented detector

For the chosen station and pollutant, the analysis computes the median m and median absolute deviation MAD. It scales MAD by 1.4826 to make it comparable to standard deviation under a normal distribution, then sets an upper threshold T = m + k × (1.4826 × MAD). A reading is flagged only when it is greater than T. The default k is 3. The detector targets high excursions; low-side anomalies are outside its current rule.

Step	Rule	Default demo value
Central baseline	Median	41.4 µg/m^3
Robust spread	1.4826 × MAD	17.79 µg/m^3
Multiplier	k	3.0
Upper threshold	Median + k × scaled MAD	94.77 µg/m^3
Flagged readings	Reading > threshold	9 of 331 valid observations

3.2 Comparison on identical readings

The comparison below applies three threshold rules to the same 331 valid Aotizhongxin PM2.5 readings. The classical mean/standard-deviation threshold is pulled upward by the large injected values. Using raw MAD without the normal-consistency scale factor is more sensitive in this example. This is a threshold comparison, not a full model benchmark: there is no labeled real-world validation set or precision/recall analysis.

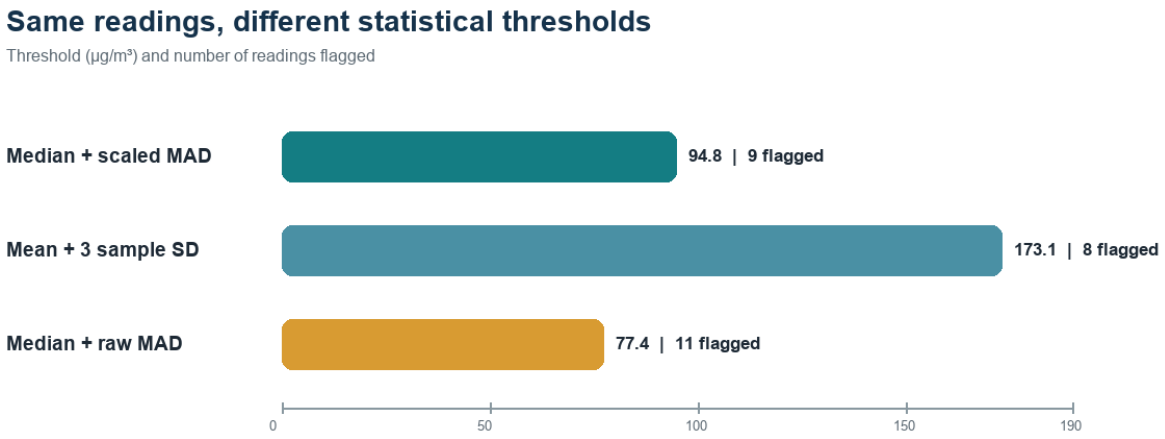


Figure 2. Statistical thresholds and resulting flags on the default synthetic example.

Method	Location / spread	Threshold (µg/m^3)	Flags	Interpretation
Median + 3 × scaled MAD	Median 41.4; scaled MAD 17.79	94.77	9	Implemented robust rule; less influenced by extremes.
Mean + 3 × sample SD	Mean 45.95; sample SD 42.39	173.13	8	Comparison only; extreme values inflate the threshold.
Median + 3 × raw MAD	Median 41.4; raw MAD 12.0	77.40	11	Comparison only; omitting scale factor increases sensitivity.

3.3 Sensitivity to k

The multiplier controls how far a reading must lie above the robust baseline before it is flagged. Increasing k makes the rule more conservative. In the demo, the count stabilizes at eight flags from k = 3.5 onward.

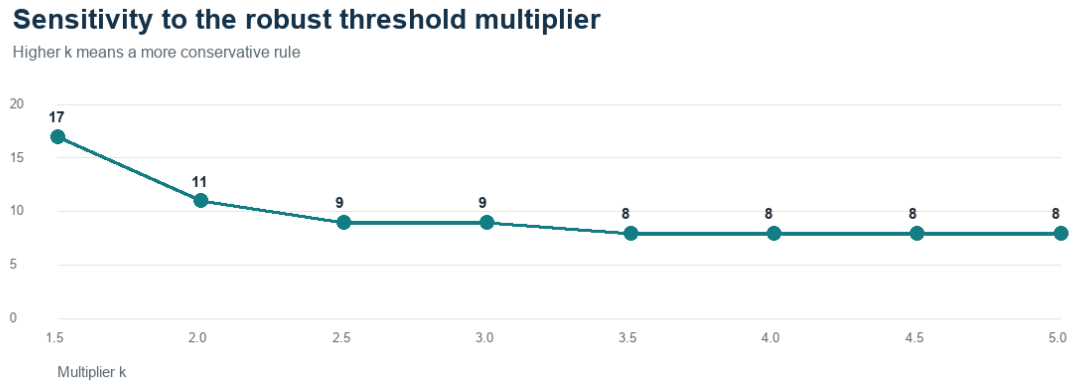


Figure 3. Sensitivity curve; counts are specific to this synthetic dataset.

4. Demonstration Results

The default synthetic run uses four stations with 336 hourly rows each, for 1,344 rows total. Aotizhongxin has 331 non-missing PM2.5 values out of 336. The injected event produces a visible excursion above the station baseline.

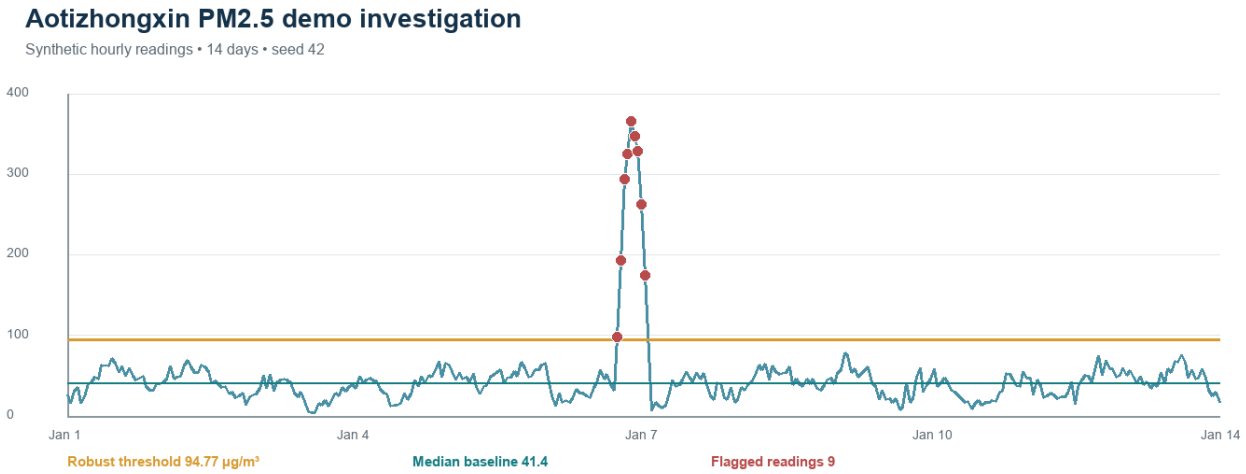


Figure 4. Hourly PM2.5 values, robust threshold, baseline and flagged samples.

Measure	Observed demo result
Station / pollutant	Aotizhongxin / PM2.5
Rows at selected station	336 hourly rows; 331 valid PM2.5 values
Median baseline	41.4 µg/m^3
Raw MAD / scaled MAD	12.0 / 17.79 µg/m^3
Threshold at k = 3	94.77 µg/m^3
Flag count / peak	9 readings / 365.5 µg/m^3
Agent outcome	ISOLATED_LOCAL_ELEVATION; HIGH_PRIORITY_INVESTIGATION; VALIDATED
Event weather summary	Average wind about 0.5 m/s; predominant ENE; no precipitation; mean temperature about −0.9 °C

Interpretation: the workflow identifies readings that depart sharply from the selected station's recent synthetic baseline and packages contextual evidence for review. The isolated-elevation label is a heuristic based on the available station values; it does not establish the event's physical cause. "Validated" is an internal rule-check status, not external scientific or regulatory certification.

5. Validation, Testing and Limitations

5.1 Validation performed

The reproducible synthetic data generation and six-agent analysis pipeline completed for the default scenario. The test suite result during the review was 25 passed and 2 failed. Both failures occurred before the UI code ran: Streamlit AppTest was invoked with a relative "app.py" path from the tests directory and resolved to a nonexistent tests/app.py. The UI server had previously started and returned HTTP 200, but an absolute-path headless AppTest run did not complete in this environment.

Check	Result	Meaning
Core pipeline	Completed	Default scenario returned analysis, context, action and critic outputs.
Automated tests	25 passed; 2 failed	Failures are AppTest file-path resolution issues; fix test path and rerun.
Streamlit page	HTTP 200 in prior launch	Server delivered a page; does not replace interactive acceptance testing.
Model quality metrics	Not available	No labeled holdout set; accuracy, precision and recall are not established.

5.2 Key limitations and improvements

Priority	Limitation found	Recommended improvement
High	Cached orchestration key omits selected dates and source/data fingerprint.	Include date bounds and stable data identity in the session key.
High	README/demo script figures are stale relative to current seed-42 output.	Regenerate demo results and update spoken notes/screenshots from one code version.
Medium	Pollutant units are labeled µg/m^3 even when CO aliases may be in mg/m^3.	Preserve source units and normalize before analysis.
Medium	Unavailable cross-station evidence can be treated as a regional case.	Handle unavailable, isolated and co-occurring evidence separately.
Medium	Critic computes a disclaimer flag but always marks the audit passed.	Make the condition meaningful and check final advisory text.
Medium	Station comparison does not include GIS distance or transport modeling.	Describe spatial labels as provisional; add location metadata and validation.
Medium	Minimum 24 observations does not ensure hourly continuity or seasonality.	Check time gaps and use longer representative baselines.
Medium	Some UI markdown renders user-controlled text as unsafe HTML.	Escape uploaded names/advisory text or use safe Streamlit widgets.
Low	Missing-data thresholds differ across components; SciPy appears unused.	Unify documented thresholds; remove unused dependency or document its role.

6. Recommendations and Future Scope

6.1 Near-term engineering priorities

- Repair the two AppTest paths and add an interaction check for station, date, source, and pollutant changes.
- Fix session-state caching so the chart and investigation always refer to the same selected data.
- Correct the README and demo script to show the verified current values.
- Preserve pollutant units through loading and convert explicitly where needed.
- Make safety/disclaimer audit checks enforceable and escape untrusted UI text.
- Separate “evidence unavailable” from “regional co-occurrence” in the response planner.

6.2 Evaluation plan

A meaningful future benchmark should use labeled real or carefully curated examples that were not used to tune the rules. Define an event label, split evaluation by time or station to reduce leakage, and report precision, recall, false-alarm rate, detection delay, and review burden alongside thresholds. Compare robust MAD, classical z-score, and any future model using the same cases and operating assumptions. Include both injected and naturally occurring events, plus missing data, sensor faults, and steady baselines.

6.3 Potential extensions

- Add rolling and seasonal baselines to account for daily and seasonal cycles.
- Add explicit gap detection and sensor calibration diagnostics before anomaly scoring.
- Use station coordinates and validated meteorological context for spatial comparison.
- Add review feedback so human analysts can label false positives and confirmed events.
- If introducing an LLM or trained model, constrain it to evidence-backed summaries and keep numeric detection independently testable.

7. Conclusion

BreatheOps AI provides a clear classroom demonstration of task decomposition, sequential agent collaboration, robust anomaly screening, and evidence-oriented reporting. Its strengths are explainability, repeatability, local operation, and separation of detection from recommended human follow-up. The current evidence supports describing it as a deterministic multi-agent statistical analytics PoC. It does not yet support claims of trained-model accuracy, regulatory assessment, causal diagnosis, or production readiness.

Appendix A. Run and reproduce

From the project directory, install the dependencies listed in requirements.txt, then start the application with:

```
streamlit run app.py
```

Choose demo data, select Aotizhongxin and PM2.5, keep k = 3.0, and run the investigation to reproduce the scenario summarized in this report.

Appendix B. Dataset source and reference

[1] Chen, S. (2017). Beijing Multi-Site Air Quality [Dataset]. UCI Machine Learning Repository. DOI: 10.24432/C5RK5G. Dataset record: https://archive.ics.uci.edu/dataset/501/beijing+multi+site+air+quality. The UCI record reports 420,768 instances, 12 monitoring sites, six air pollutants, six meteorological variables, hourly

observations from 1 March 2013 to 28 February 2017, and missing values denoted as NA. The project archive was checked locally and contains 12 station CSVs of 35,064 rows each.

Appendix C. Project components

File / component	Role
app.py	Streamlit user interface and interactive report presentation.
breatheops/data_loader.py	Input normalization, timestamp parsing, duplicate and missingness handling.
breatheops/demo_data.py	Seeded synthetic demo generator.
breatheops/analysis.py	Robust baseline and anomaly detection.
breatheops/orchestrator.py	Shared-context sequential workflow.
breatheops/agents/	Planner, data-quality, detection, context, response and critic agent classes.
tests/	Unit, robustness, and Streamlit AppTest coverage.